

Loan Approval Prediction

Importing libraries

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

uploading DataSet

```
In [5]: df=pd.read_csv('LoanApprovalPrediction.csv')
df
```

```
Out[5]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001002	Male	No	0.0	Graduate	No	5849
1	LP001003	Male	Yes	1.0	Graduate	No	4583
2	LP001005	Male	Yes	0.0	Graduate	Yes	3000
3	LP001006	Male	Yes	0.0	Not Graduate	No	2583
4	LP001008	Male	No	0.0	Graduate	No	6000
...	...	...	...	...	...	...	...
593	LP002978	Female	No	0.0	Graduate	No	2900
594	LP002979	Male	Yes	3.0	Graduate	No	4106
595	LP002983	Male	Yes	1.0	Graduate	No	8072
596	LP002984	Male	Yes	2.0	Graduate	No	7583
597	LP002990	Female	No	0.0	Graduate	Yes	4583

598 rows × 13 columns



```
In [6]: df.shape
```

```
Out[6]: (598, 13)
```

```
In [7]: df.describe()
```

Out[7]:

	Dependents	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
count	586.000000	598.000000	598.000000	577.000000	584.000000
mean	0.755973	5292.252508	1631.499866	144.968804	341.917808
std	1.007751	5807.265364	2953.315785	82.704182	65.205994
min	0.000000	150.000000	0.000000	9.000000	12.000000
25%	0.000000	2877.500000	0.000000	100.000000	360.000000
50%	0.000000	3806.000000	1211.500000	127.000000	360.000000
75%	1.750000	5746.000000	2324.000000	167.000000	360.000000
max	3.000000	81000.000000	41667.000000	650.000000	480.000000

Large differences between mean and median of all the features. Presence of outliers as there is a huge difference in min and max values Data is positively skewed.

In []:

In [8]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 598 entries, 0 to 597
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                598 non-null    object
1   Gender                 598 non-null    object
2   Married                598 non-null    object
3   Dependents             586 non-null    float64
4   Education              598 non-null    object
5   Self_Employed          598 non-null    object
6   ApplicantIncome        598 non-null    int64
7   CoapplicantIncome      598 non-null    float64
8   LoanAmount             577 non-null    float64
9   Loan_Amount_Term       584 non-null    float64
10  Credit_History          549 non-null    float64
11  Property_Area           598 non-null    object
12  Loan_Status             598 non-null    object
dtypes: float64(5), int64(1), object(7)
memory usage: 60.9+ KB
```

In [9]: `## dropping the 'Loan_id' col as this is not required for analysis`

```
df=df.drop(['Loan_ID'],axis=1)
df.head(5)
```

Out[9]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplicant
0	Male	No	0.0	Graduate	No	5849	
1	Male	Yes	1.0	Graduate	No	4583	
2	Male	Yes	0.0	Graduate	Yes	3000	
3	Male	Yes	0.0	Not Graduate	No	2583	
4	Male	No	0.0	Graduate	No	6000	

In []:

Exploratory Data Analysis

Visualising all the unique values in columns using barplot. This will simply show which value is dominating as per our dataset.

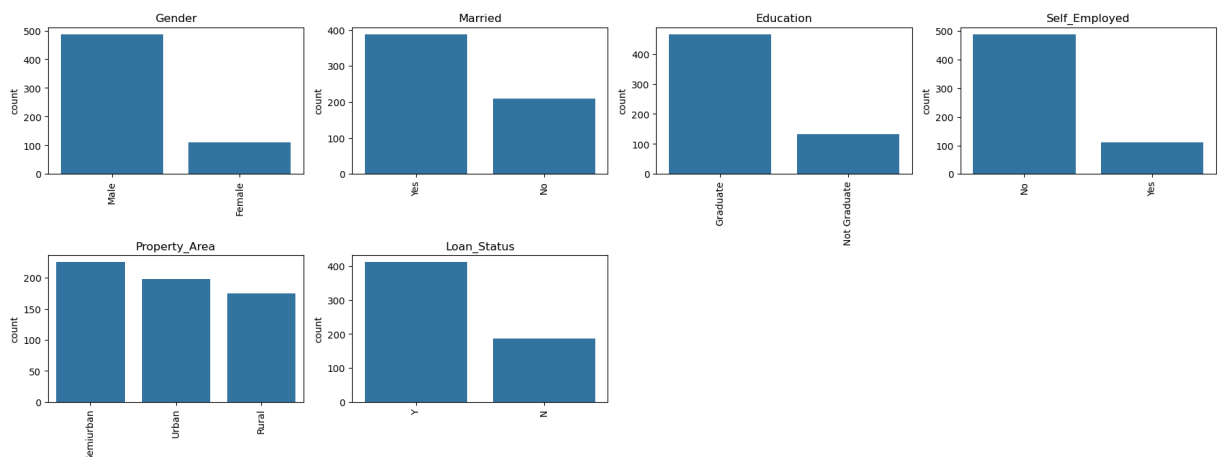
```
In [11]: cat_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()
print(cat_cols)
```

```
['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area', 'Loan_Status']
```

```
In [12]: obj = (df.dtypes == 'object')
object_cols = list(obj[obj].index)
plt.figure(figsize=(18, 36))
index = 1

for col in object_cols:
    y = df[col].value_counts()
    plt.subplot(11, 4, index)
    plt.title(col, fontsize=12)
    sns.barplot(x=list(y.index), y=y)
    plt.xticks(rotation=90)
    index += 1

plt.tight_layout()
plt.show()
```



Since all the categorical Columns are binary, we can convert them into numerical columns using 'Labelencoder'

```
In [13]: le=LabelEncoder()

for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

```
In [14]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 598 entries, 0 to 597
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                 598 non-null   int32
1   Married                598 non-null   int32
2   Dependents             586 non-null   float64
3   Education              598 non-null   int32
4   Self_Employed          598 non-null   int32
5   ApplicantIncome        598 non-null   int64
6   CoapplicantIncome      598 non-null   float64
7   LoanAmount             577 non-null   float64
8   Loan_Amount_Term       584 non-null   float64
9   Credit_History         549 non-null   float64
10  Property_Area          598 non-null   int32
11  Loan_Status            598 non-null   int32
dtypes: float64(5), int32(6), int64(1)
memory usage: 42.2 KB
```

```
In [30]: df.head(5)
```

```
Out[30]:
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplicant
0	1	0	0.0	0	0	5849	
1	1	1	1.0	0	0	4583	
2	1	1	0.0	0	1	3000	
3	1	1	0.0	1	0	2583	
4	1	0	0.0	0	0	6000	

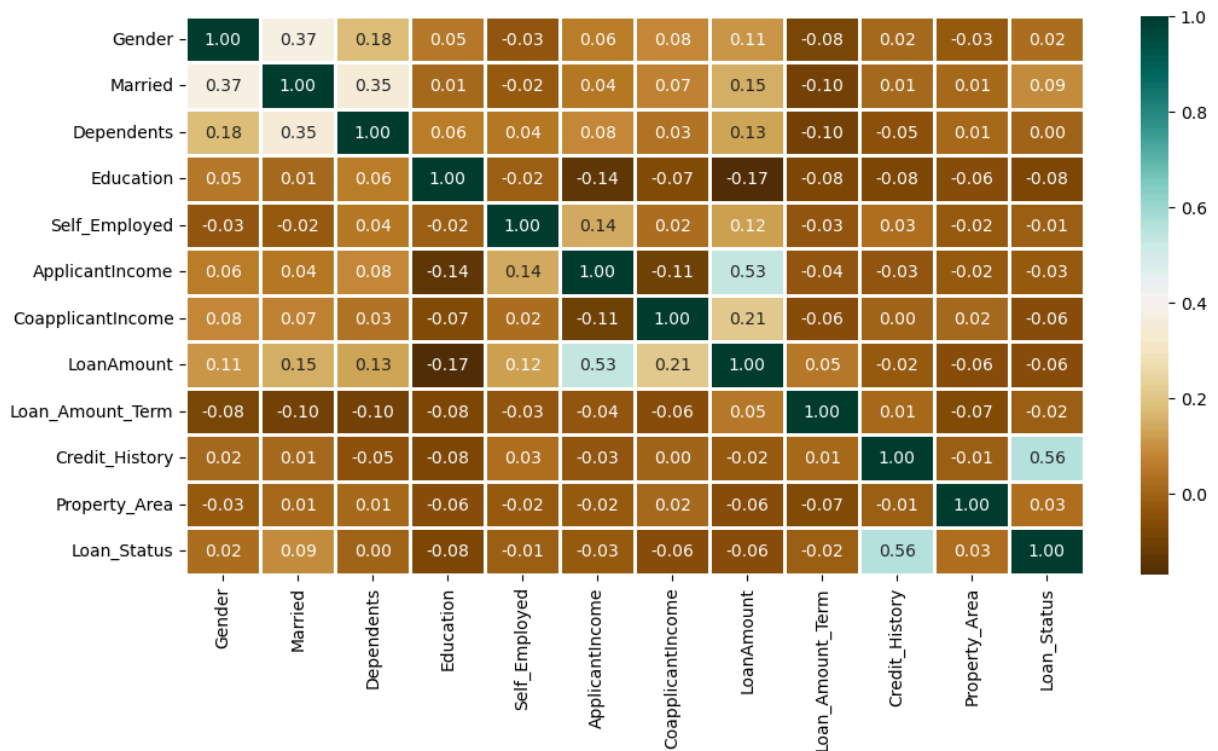
```
In [ ]:
```

Heatmap - to understand the correlation between each cols

```
In [17]: plt.figure(figsize=(12,6))

sns.heatmap(df.corr(),cmap='BrBG',fmt='.2f',linewidths=2,annot=True)
```

```
Out[17]: <Axes: >
```



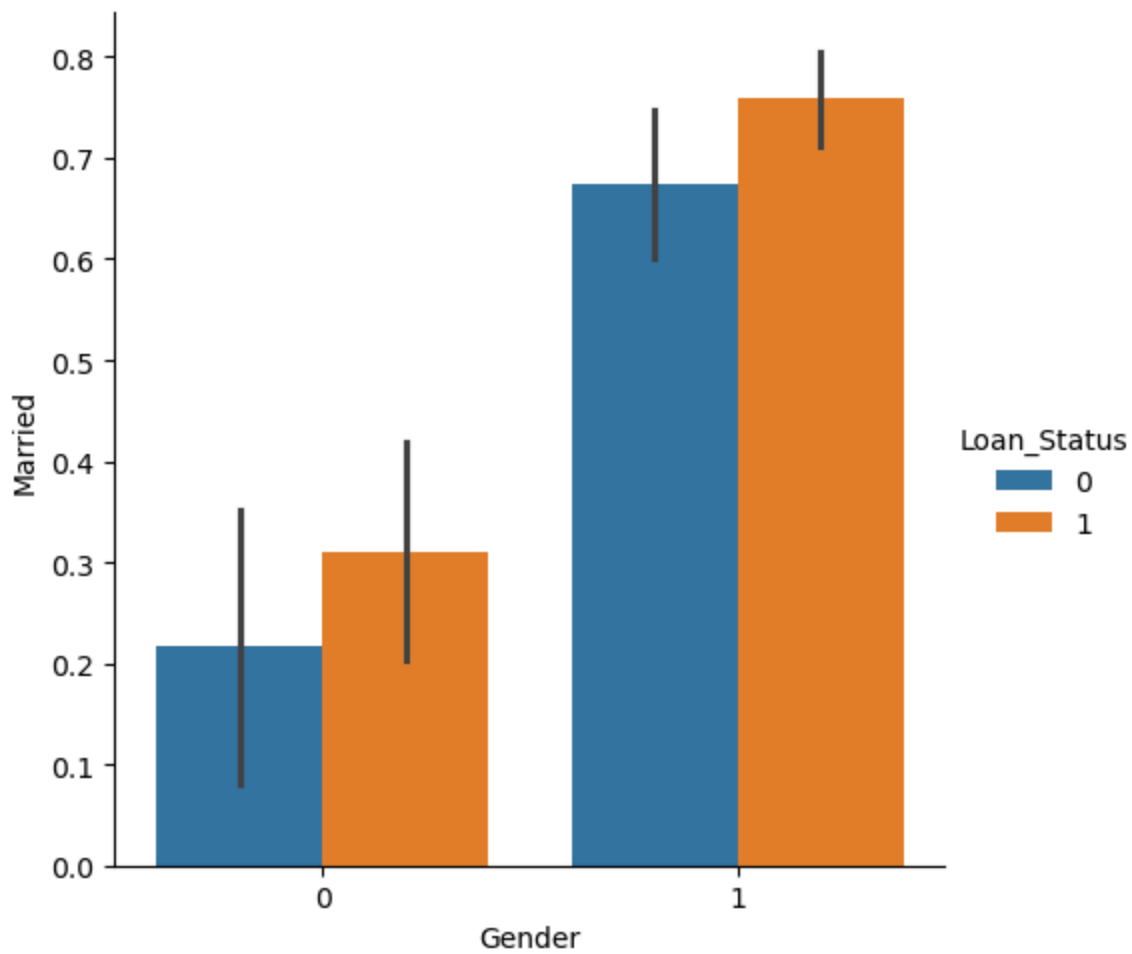
The above heatmap shows the correlation (0.53) between Loan Amount and ApplicantIncome. It also shows that Credit_History has a high impact(0.56) on Loan_Status.

Catplot

To visualize the plot for the Gender and Marital Status of the applicant.

```
In [32]: sns.catplot(x="Gender", y="Married",
                    hue="Loan_Status",
                    kind="bar",
                    data=df)
```

```
Out[32]: <seaborn.axisgrid.FacetGrid at 0x1a44bb70ce0>
```



Handling missing Values

```
In [34]: for col in df.columns:
          df[col] = df[col].fillna(df[col].mean())

          df.isna().sum()
```

```
Out[34]: Gender          0
          Married        0
          Dependents     0
          Education      0
          Self_Employed  0
          ApplicantIncome 0
          CoapplicantIncome 0
          LoanAmount     0
          Loan_Amount_Term 0
          Credit_History  0
          Property_Area   0
          Loan_Status     0
          dtype: int64
```

There are no missing values in the dataset

```
In [ ]:
```

Data Splitting

```
In [36]: X = df.drop(['Loan_Status'],axis=1)
y = df['Loan_Status']
X.shape, y.shape

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.4, random_stat
X_train.shape, X_test.shape, y_train.shape, y_test.shape

Out[36]: ((358, 11), (240, 11), (358,), (240,))
```

```
In [ ]:
```

Model Training

Being a classification problem using KNeighborsClassifiers, RandomForestClassifiers,Support Vector Classifiers (SVC) and Logistics Regression

```
In [40]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
import warnings
warnings.filterwarnings('ignore')
```

```
In [42]: knn = KNeighborsClassifier(n_neighbors=3)
rfc = RandomForestClassifier(n_estimators = 7,
                             criterion = 'entropy',
                             random_state =7)

svc = SVC()
lc = LogisticRegression()

# making predictions on the training set
for clf in (rfc, knn, svc,lc):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_train)
    print("Accuracy score of ",
          clf.__class__.__name__,
          "=",100*metrics.accuracy_score(y_train,
                                          y_pred))
```

```
Accuracy score of RandomForestClassifier = 98.04469273743017
Accuracy score of KNeighborsClassifier = 78.49162011173185
Accuracy score of SVC = 68.71508379888269
Accuracy score of LogisticRegression = 80.44692737430168
```

```
In [ ]:
```

Prediction on test data(Before SMOTE)

```
In [46]: before_smote = {}
```

```
after_smote = {}
```

```
In [48]: for clf in (rfc, knn, svc, lc):
          clf.fit(X_train, y_train)
          y_pred = clf.predict(X_test)
          accuracy = 100*metrics.accuracy_score(y_test, y_pred)
          before_smote [clf.__class__.__name__] = accuracy
          print (clf.__class__.__name__, '=', accuracy)
```

```
RandomForestClassifier = 82.5
KNeighborsClassifier = 63.74999999999999
SVC = 69.16666666666667
LogisticRegression = 80.83333333333333
```

Random Forest Classifier is giving the best accuracy of 82% for the testing dataset

```
In [ ]:
```

```
In [50]: from sklearn.metrics import classification_report

          print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.81	0.48	0.60	73
1	0.81	0.95	0.87	167
accuracy			0.81	240
macro avg	0.81	0.72	0.74	240
weighted avg	0.81	0.81	0.79	240

Accuracy is 0.81 which means that the model correctly predicts the output 81% of the time which is high. However the breakdown by class shows a significant performance gap. The model performs very well for class 1 (Majority class - 167 samples) with a Recall of 0.95, identifying 95% of all actual "1" cases. However the model struggles for class 0 (Minority class - 73 samples) with a Recall of only 0.48 missing more than half (52%) of the total. The precision is 0.81 so when it does guess "0", it's usually correct, but it is far too conservative in making that prediction. Therefore, the model struggles to correctly identify negatives/non-approved loans.

```
In [52]: print(y_test.value_counts())
```

```
Loan_Status
1    167
0     73
Name: count, dtype: int64
```

```
In [ ]:
```

Handling Imbalance in the Train Dataset

Dataset is imbalanced (167/73 split) making the model biased towards class 1 resulting in high accuracy but poor sensitivity for class 0

```
In [54]: from imblearn.over_sampling import SMOTE

          smote = SMOTE(random_state=7)

          X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```



```
# Check class distribution
print("Before SMOTE:")
print(y_train.value_counts())

print("\nAfter SMOTE:")
print(y_train_smote.value_counts())
```

Before SMOTE:

Loan_Status

1 244

0 114

Name: count, dtype: int64

After SMOTE:

Loan_Status

0 244

1 244

Name: count, dtype: int64

```
In [56]: for clf in (rfc, knn, svc, lc):
          clf.fit(X_train_smote, y_train_smote)
          y_pred = clf.predict(X_test)
          accuracy = 100*metrics.accuracy_score(y_test, y_pred)
          after_smote [clf.__class__.__name__] = accuracy
          print (clf.__class__.__name__, '=', accuracy)
```

RandomForestClassifier = 73.33333333333333

KNeighborsClassifier = 55.41666666666667

SVC = 61.66666666666667

LogisticRegression = 71.66666666666667

```
In [58]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.53	0.66	0.59	73
1	0.83	0.74	0.78	167
accuracy			0.72	240
macro avg	0.68	0.70	0.69	240
weighted avg	0.74	0.72	0.72	240

In []: Accuracy reduced to 0.72 after smote which is very common after balancing data. After Minority class prediction improves resulting in a healthier and fairer model.

For Class 0 (Minority class - 73 samples) Precision is 0.53 which means 53% are actual. Recall is 0.66 meaning 66% have been correctly identified. SMOTE improved minority class.

For Class 1 (Major class - 167 samples) Precision is 0.83 which means 83% are actual. Recall is 0.74 meaning 74% have been correctly identified.

Accuracy Comparison Table

```
In [60]: accuracy_table = pd.DataFrame({
        'Before SMOTE': before_smote,
        'After SMOTE': after_smote
    })

accuracy_table.reset_index(inplace=True)
accuracy_table.rename(columns={'index': 'Model'}, inplace=True)

accuracy_table['Change'] = (
    accuracy_table['After SMOTE'] -
    accuracy_table['Before SMOTE'])

accuracy_table = accuracy_table.round(2)

print(accuracy_table.to_string(index=False))
```

	Model	Before SMOTE	After SMOTE	Change
	RandomForestClassifier	82.50	73.33	-9.17
	KNeighborsClassifier	63.75	55.42	-8.33
	SVC	69.17	61.67	-7.50
	LogisticRegression	80.83	71.67	-9.17

Accuracy decreased after SMOTE, which is normal as prior to SMOTE data was: -Dataset was imbalanced. -Majority class (1) had many more samples than minority class (0). -Models became biased toward predicting the majority class. -Accuracy looked artificially high. After SMOTE dataset: - The model becomes less biased. - Minority class prediction improves. - Recall for class 0 generally increases. - Model becomes fairer and more realistic. In imbalanced datasets, Accuracy alone is not enough. Precision, Recall, F1-score are more important. Modelwise Analysis 1. Random Forest Classifier - 82.50(before) 73.33 (after) with a change of -9.17 It is still the best performing model as it handles imbalance better than other models. Accuracy decreased because it now tries to classify minority class correctly. 2. Logistic Regression - 80.83(before) 71.67(after) with a change of -9.17 It gives a Good stable performance. Works well after balancing and is more interpretable. 3. KNeighborsClassifier - 63.75(before) 55.42(after) with a change of -8.33 KNN is sensitive to synthetic samples created by SMOTE. Synthetic neighbors may confuse the model. Therefore, KNeighborsClassifier is Not ideal here 4. SVC - 69.17(before) 61.67(after) with a change of -7.50 SVC performance dropped heavily. SMOTE may have introduced overlapping boundary points. SVC struggles when class boundaries become noisy. Weakest model after SMOTE. Conclusion: Before applying SMOTE, the models achieved higher accuracy due to class imbalance in the dataset. After applying SMOTE, the dataset became balanced, resulting in slightly lower accuracy but improved minority class prediction and overall model fairness. Among all models, Random Forest Classifier performed best both before and after SMOTE. Therefore, Misclassifying rejected loans as approved can be risky and Minority class prediction matters.

In []:

In []:

In []:

In []:

In []:

In []: